

Software Requirements Specification - Milestone 2

1. Scop

Aplicatia SQL Code Generator este o aplicatie client-server care primeste o schema ER in format JSON, genereaza cod SQL `CREATE TABLE` pentru tabele si relatii, valideaza batch-uri `INSERT` inainte ca acestea sa fie aplicate intr-o baza de date reala si ofera un canal separat pentru administrarea serverului.

Versiunea Milestone 2 extinde rezultatul din Milestone 1 printr-un server functional, client ordinar C, client ordinar alternativ Python, client de administrare ncurses, protocol binar, transfer de fisiere in ambele sensuri, coada FIFO de procesare, job tracking si rapoarte administrative observabile.

2. Domeniu si Actori

Actorul ADMIN se conecteaza pe portul de administrare si executa rapoarte sau operatii de mentenanta. Actorul REMOTE/IN este clientul ordinar care se conecteaza prin socket INET si foloseste serviciile de upload schema, generare SQL, download SQL si validare `INSERT`. Serverul gestioneaza ambele tipuri de clienti, coada de procesare, validarea datelor si starea comuna a aplicatiei.

Aplicatia foloseste `cJSON` pentru parsarea diagramei ER si `libpg_query` pentru analiza sintactica PostgreSQL a instructiunilor SQL. Procesarea datelor se face in C; clientul Python este doar o implementare alternativa de client ordinar pe acelasi protocol binar.

3. Ipoteze si Constrangeri

Conexiunile ordinare folosesc TCP/INET, iar administrarea este expusa pe un canal separat. Serverul nu porneste scripturi Python sau Java pentru generare sau validare. Formatul ER acceptat urmeaza structura din `examples/er_schema.json`, cu obiect `tables`, coloane si optional `relations`. Daca proiectul va primi ulterior o componenta web services, aceasta trebuie proiectata pe gSOAP/SOAP.

4. Cerinte Functionale Pentru Clientul Ordinar

Clientul ordinar trebuie sa se conecteze prin `OP_CONNECT`, iar serverul trebuie sa ii aloce un `client_id` unic. Operatiile trimise inainte de conectare trebuie respinse explicit. Sesiunea se inchide controlat prin `OP_BYE`.

Clientul trebuie sa poata incarca o schema ER JSON catre server folosind `OP_UPLOAD_BEGIN`, `OP_UPLOAD_CHUNK` si `OP_UPLOAD_END`. Transferul client-server se face in bucati de cel mult `SQLCG_FILE_CHUNK`, adica 64 KB. Dupa finalizarea uploadului, serverul trebuie sa parseze schema cu `cJSON` si sa actualizeze modelul intern.

Clientul trebuie sa poata cere generarea codului SQL prin `OP_GENERATE_SQL`. Serverul trebuie sa genereze instructiuni `CREATE TABLE` pentru tabele, chei primare, constrangeri `UNIQUE`, constrangeri `NOT NULL` si chei externe. Clientul trebuie sa poata descarca SQL-ul generat ca fisier prin `OP_DOWNLOAD_SQL`, iar transferul server-client trebuie sa foloseasca `OP_DOWNLOAD_BEGIN`, `OP_DOWNLOAD_CHUNK` si `OP_DOWNLOAD_END`.

Clientul trebuie sa poata trimite batch-uri **INSERT** prin **OP_VALIDATE_INSERT**. Serverul trebuie sa valideze sintactic batch-ul cu **libpg_query**, apoi sa il valideze semantic fata de schema ER si randurile deja acceptate. Doar batch-urile valide se aplica in memoria serverului. Erorile raportate trebuie sa acopere tabela inexistentă, coloana inexistentă, număr gresit de valori, incalcări **NOT NULL**, duplicate pe **PRIMARY KEY**, duplicate pe **UNIQUE** si foreign key lipsa.

Operatiile de procesare trebuie sa primeasca un **job_id**. Clientul trebuie sa poata cere sincron starea procesarii prin **status [job_id|last]** si rezultatul procesarii prin **result [job_id|last]**. Sistemul trebuie sa includa si un client ordinar alternativ in Python, interoperabil cu protocolul IN folosit de clientul C.

5. Cerinte Functionale Pentru Clientul Admin

Adminul trebuie sa se autentifice pe port separat prin **OP_ADMIN_LOGIN**. Sistemul trebuie sa permita un singur administrator conectat simultan si trebuie sa inchida conexiunea admin dupa un timeout de inactivitate configurabil.

Adminul trebuie sa poata cere rapoarte despre clientii conectati, comenzile totale, comenzile **INSERT**, comenzile esuate, comenzile anulate, comanda activa, durata medie de executie, istoricul recent, tabelele incarcate, numărul de randuri memorate, adancimea cozii FIFO si joburile recente.

Adminul trebuie sa poata deconecta fortat un client ordinar dupa **client_id**, sa ceara anularrea comenzii curente sau urmatoare pentru un client si sa blocheze accesul dinspre un IP sau domeniu. Clientul admin trebuie sa ofere atat mod interactiv ncurses, cat si mod neinteractiv prin argumente CLI, pentru demonstratii automate.

6. Cerinte Nefunctionale

Serverul trebuie sa accepte mai multi clienti ordinari simultan si sa foloseasca fire de executie separate pentru interfata IN, interfata admin UX si workerul FIFO. Cererile clientilor ordinari trebuie puse intr-o coada FIFO comuna, protejata prin mutex si condition variable. Canalul admin trebuie sa fie sincron si separat de canalul IN.

Joburile de procesare trebuie sa aiba starile **QUEUED**, **RUNNING**, **DONE** si **ERROR**. Pentru validarea fiecarui batch **INSERT**, serverul trebuie sa creeze un proces copil si sa trimita rezultatul validarii catre parinte prin pipe. Starea necesara validarii trebuie sa fie disponibila in memorie partajata **mmap**.

Buildul C trebuie sa treaca fara warning-uri cu **-Wall -Wextra -Wpedantic -Werror** si trebuie compilat cu **-std=c11 -D_POSIX_C_SOURCE=200809L**. Repository-ul trebuie sa includa **.clang-tidy** cu checks pentru analyzer, bugprone, cert, concurrency, misc, performance, portability si readability tratate ca erori. Pentru profilare trebuie sa existe tinte **ASan/LSan/UBSan, TSan, Valgrind Mem-check** si **Valgrind Helgrind**.

Protocolul trebuie sa foloseasca header fix binar cu campurile **msg_size**, **client_id**, **op_id** si **flags**, transmise in network byte order. Payload-ul maxim trebuie limitat de **SQLCG_MAX_PAYLOAD**, iar transferul de fisiere trebuie sa functioneze prin chunking pentru fisiere modeste, pana la cateva sute de MB. Configurarea porturilor si timeoutului admin trebuie sa se poata face prin **config.cfg**, variabile de mediu si argumente CLI. Serverul trebuie sa scrie evenimente demonstrabile in **logs/server.log**.

7. Interfete Externe

Protocolul este definit de structura `MsgHeader`, formata din `msg_size`, `client_id`, `op_id` si `flags`. Headerul este urmat de payload binar de lungime exacta `msg_size`. Operatiile relevante sunt definite in `include/protocol.h` si acopera conectare, upload, download, generare SQL, validare `INSERT`, interogare joburi, rapoarte admin si operatii admin active.

Configurarea serverului are prioritate in ordinea valori implicite, fisier `config.cfg`, variabile de mediu si argumente CLI. Valorile implicite sunt port IN 18081, port admin 18082 si timeout admin 60 de secunde.

8. Imbunatatiri Obligatorii Fata de Milestone 1

SRS-ul este actualizat fata de Milestone 1 si include explicit imbunatatirile rezultate dupa prima etapa. In implementarea curenta exista operatii active de administrare pentru deconectare client, anulare comanda si blocare IP/domeniu; rapoarte admin extinse pentru clienti, comenzi, durata medie, istoric, tabele, coada si joburi; client admin atat in mod ncurses, cat si in mod neinteractiv; client ordinar alternativ Python; transfer bidirectional de fisiere; job tracking cu `job_id`, stari si interogare `status/result`; coada FIFO comuna; validare `INSERT` in proces copil; memorie partajata pentru starea de validare; logging; configurare prin fisier, mediu si CLI; tinte de analiza si profilare; integrare `cJSON` si `libpg_query`.

9. Mapare Cerinte - Implementare

Protocolul este implementat in `include/protocol.h` si `src/protocol.c`. Modelul ER, generarea SQL, validarea si rapoartele administrative sunt implementate in `include/model.h` si `src/model.c`. Serverul, coada FIFO, joburile, administrarea si logging-ul sunt implementate in `src/server.c`. Clientul ordinar C este in `src/client.c`, clientul admin este in `src/admin_client.c`, iar clientul ordinar Python este in `clients/python_client.py`. Datele de test sunt in `examples/er_schema.json` si in fisierele `examples/inserts_*.sql`. Buildul, analiza si profilarea sunt definite in `Makefile` si `.clang-tidy`.

10. Criterii de Acceptare

Solutia este acceptata daca `make` construieste executabilele `server`, `client` si `admin`, iar `make docs` regenereaza `docs/SRS.pdf` si `docs/SDD.pdf` din sursele Markdown. Clientul C trebuie sa poata incarca schema ER, genera SQL, descarca SQL si valida fisiere `INSERT`. Clientul Python trebuie sa poata executa acelasi flux minim pe protocolul binar. Adminul trebuie sa poata produce rapoartele cerute si sa execute operatiile active. Erorile de constrangere trebuie raportate inainte de aplicarea datelor invalide, iar evenimentele importante trebuie sa apara in `logs/server.log`.